

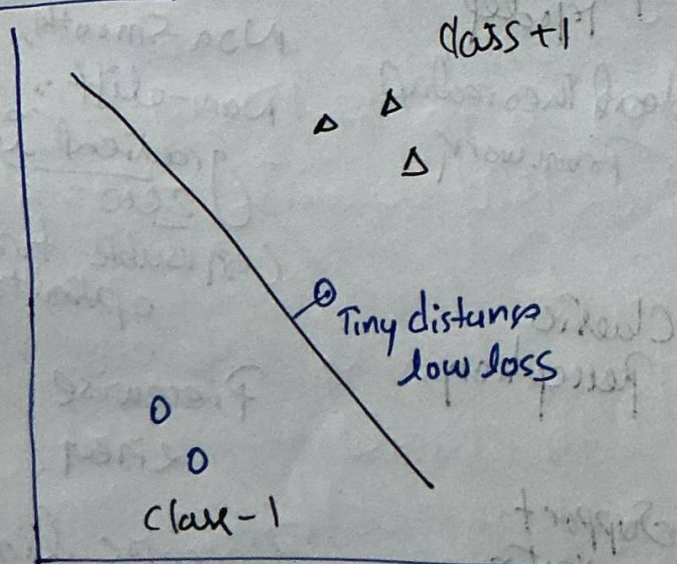
Perceptron Loss Function

Limitation ✖ The perceptron can only see errors,
Not progress

$$W_{\text{new}} = W_{\text{old}} + \eta yx$$

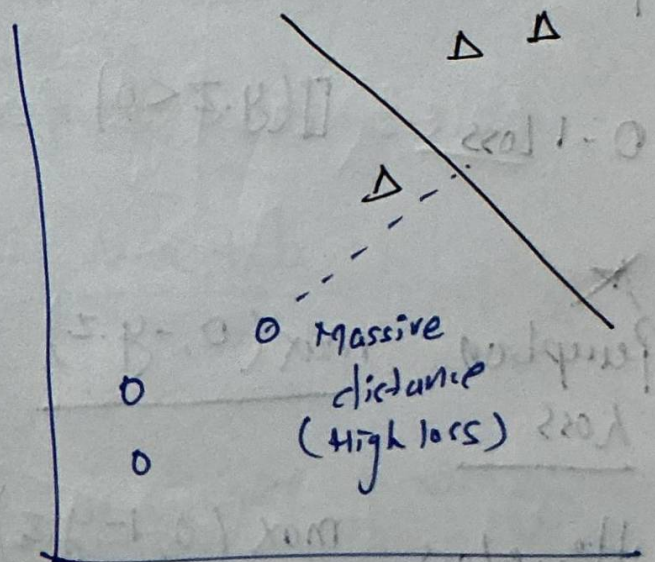
Purely binary $\rightarrow$ It checks if a prediction is right or wrong.

Model A: 1 Error



(Model is very close
Boundary to correct)

Model B: 4 Error



(Boundary is completely
backward).

when choosing a loss function for binary linear
Classification architectures (like Perceptron)

↓
Because the true indicator loss
(0-1) Loss
has gradient of zero everywhere

↓
use continuous
approx called
Surrogate loss
function.

$$Z = W^T X + b$$

Loss
function

Math
form

Primary
optimization
model

Property.

0-1 Loss

$$\mathbb{I}(y \cdot z < 0)$$

Ideal Theoretical
Formwork

Non-smooth,
Non-diff.,
gradient is
zero
(usable for
optimization)

★
Perceptron
loss

$$\max(0, -y \cdot z)$$

Classic
Perceptron

Piecewise
linear.

Hinge loss

$$\max(0, 1 - y \cdot z)$$

Support
Vector
Machine

Piecewise linear
with a margin
buffer zone.
forces max class
separation.

Logistic
Loss

$$\log(1 + \exp(-y \cdot z))$$

Logistic
Regression/
MLPs

Probabilistic
interpretation.

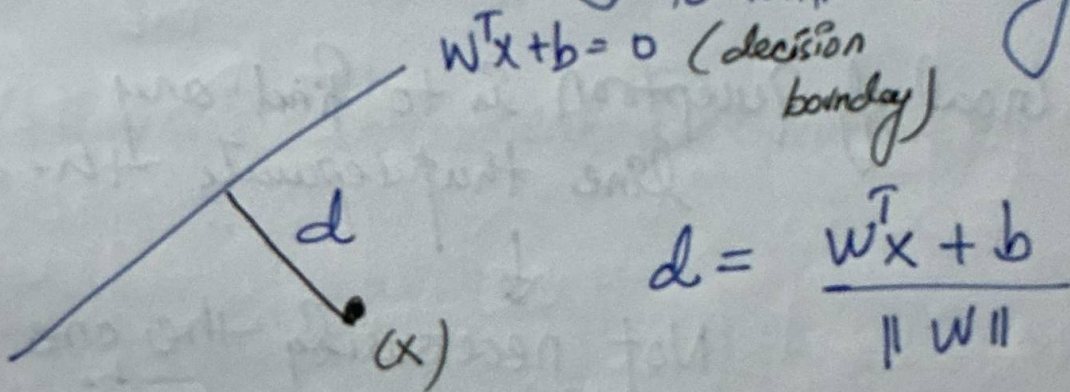
Perceptron loss function:-

we want a continuous loss function $L(w, b)$ for a single training sample

- 1) The geometric hyperplane equation separating two classes (x, y) where $y \in \{-1, +1\}$.

$$W^T x + b = 0$$

- 2) Shortest perpendicular Euclidean distance 'd' from any arbitrary feature point x to this boundary hyperplane



$$d = \frac{W^T x + b}{\|W\|}$$

- 3) To factor whether prediction is right or wrong, use Signed distance

$$d_{\text{signed}} = \frac{y(W^T x + b)}{\|W\|}$$

- If point i is Correctly classified,

$$(y) \quad (w^T x + b)$$

Same Sign

$$\Rightarrow \boxed{d_{\text{signed}} \geq 0}$$

- If point i is misclassified.

$$(y) \quad (w^T x + b)$$

Misclassified

$$\Rightarrow \boxed{d_{\text{signed}} < 0}$$

NOTE: Goal of Perceptron is to find any line that separates the data



Not necessarily the one with clearest geometric

Clearance Margin.

Therefore, dividing by $\|w\|$ will add unnecessary computational

Complexity.

$$\boxed{\text{Functional margin} = y(w^T x + b)}$$

Only penalize if thing go wrong i.e.
 $y(w^T x + b) < 0$

So,
$$\boxed{L(w, b) = \max(0, -y(w^T x + b))}$$

perceptron loss function

Computing Subgradient Derivation for Optimization:

Condition 1:- The point is correctly classified

when $y(w^T x + b) \geq 0 \Rightarrow \max(0, -ve)$

actual label $\rightarrow$ Predicted label $\rightarrow$ $L = 0$

gradient

$$\boxed{\nabla_w L = 0}$$

Condition 2 - The point is misclassified

$$\text{When } y(\omega^T x + b) < 0 \Rightarrow \max(0, \text{tve})$$

i.e.

$$\mathcal{L} = -y(\omega^T x + b)$$

$$\mathcal{L} = -y\omega^T x - yb$$

↓
tve value
↓
Error

$$\text{so, } \nabla_{\omega} \mathcal{L} = \frac{\partial}{\partial \omega} (-y\omega^T x - yb) \\ = -yx$$

Thus,

$$\nabla_{\omega} \mathcal{L} = \begin{cases} 0 & \text{if } y(\omega^T x + b) \geq 0 \\ -yx & \text{if } y(\omega^T x + b) < 0 \end{cases}$$

Connecting the loss Derivation
back to Rosenblatt's Loop

Put into "Standard Stochastic
Gradient Descent (SGD)"
Update rule.

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \nabla_{\text{old}} \mathcal{L}$$

If Model makes
No mistake.
i.e. $y(w^T x + b) > 0$

$$\nabla_{\text{old}} \mathcal{L} = 0$$

$$W_{\text{new}} = W_{\text{old}}$$

No update

If Model makes
mistakes.

$$\text{i.e. } y(w^T x + b) < 0$$

$$\nabla_{\text{old}} \mathcal{L} = -y x$$

$$\text{so, } W_{\text{new}} = W_{\text{old}} - \eta(-y x)$$

$$W_{\text{new}} = W_{\text{old}} + \eta y x$$

Example 2

X_1	X_2	Y
x_1^1	x_2^1	y_1
x_1^2	x_2^2	y_2

features: $X_{\text{raw}}^{(i)} = [x_1^{(i)}, x_2^{(i)}]^T$

Weights $W_{\text{raw}} = [w_1, w_2]^T$

Bias = b .

Target label
 $y^{(i)} \in \{-1, +1\}$

$$Z^{(i)} = w_1 x_1^{(i)} + w_2 x_2^{(i)} + b$$

Augment
Input
Vector

$$X^{(i)} = \begin{bmatrix} 1 \\ x_1^{(i)} \\ x_2^{(i)} \end{bmatrix} \in \mathbb{R}^3$$

Augment
weight Vector

$$W = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix} \in \mathbb{R}^3$$

Design matrix (X)

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times 3}$$

Bias intercept.

Features

Training set - 1

Training set - n

Target vector (y)

$$y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n$$

So,

$$Z = XW$$

$$Z = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ \vdots & x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} \end{bmatrix} \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$$

$n \times 3$ 3×1

$$Z = \begin{bmatrix} b + w_1 x_1^{(1)} + w_2 x_2^{(1)} \\ b + w_1 x_1^{(2)} + w_2 x_2^{(2)} \\ \vdots \\ b + w_1 x_1^{(n)} + w_2 x_2^{(n)} \end{bmatrix}$$

$$Z = XW = \begin{bmatrix} w^T x^{(1)} \\ w^T x^{(2)} \\ \vdots \\ w^T x^{(n)} \end{bmatrix}$$

Loss function

$$L(w) = \frac{1}{n} \sum_{i=1}^n \max(0, -y^{(i)}(w^T x^{(i)}))$$

→ Points that are correctly classified return a loss value = 0

→ This allows to strip away the piecewise $\max(0, \cdot)$ operation

↓
and rewrite the total active loss purely as the sum of our misclassified distance.

~~$$L(w) = -\frac{1}{n} \sum_{i=1}^n \max(0, y^{(i)} (w^T x^{(i)}))$$~~

$$L(w) = -\frac{1}{n} \sum_{i \in \mathcal{U}(w)} y^{(i)} (w^T x^{(i)})$$

↓
active index tracking

(keeps track of only the points that currently misclassified by our weights!)

To minimize

our global loss function:

Using Stochastic Gradient Descent (SGD)

↓
Need to find direction of slope of our cost function.

$$\nabla_w L(w) = \frac{\partial}{\partial w} \left(-\frac{1}{n} \sum_{i \in \mathcal{U}(w)} y^{(i)} (w^T x^{(i)}) \right)$$

$$\nabla_w L(w) = -\frac{1}{n} \sum_{i \in \mathcal{U}(w)} y^{(i)} \frac{\partial}{\partial w} (w^T x^{(i)})$$

$$\boxed{\nabla_w L(w) = -\frac{1}{n} \sum_{i \in \mathcal{U}(w)} y^{(i)} x^{(i)}}$$

To update our weights, take a step in the opposite direction of this error slope, scaled by learning rate (η).

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \nabla_w L(w)$$

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \left[-\frac{1}{n} \sum_{i \in \mathcal{U}(w)} y^{(i)} x^{(i)} \right]$$

$$\boxed{w_{\text{new}} = w_{\text{old}} + \frac{\eta}{n} \sum_{i \in \mathcal{U}(w)} y^{(i)} x^{(i)}}$$

Finding the final weights:-

1) The Forward Pass:-

The Model Processes the entire
design matrix (Xw)

and Identifies all misclassified
Sample indices

2) The Collective Pull:- $(\mathcal{M}(w))$

Each misclassified sample vector $x^{(i)}$
acts like physical force.

→ Scaled by its true

3) The Parameter
shift:-

Label
direction $y^{(i)}$

The algorithm adds all of
these corrective forces
together and shifts entire unified
weight

$$w = [b, w_1, w_2]^T$$

Simultaneously in single step.

"The algorithm repeats this optimization pass epoch after epoch.

The moment the misclassified tracking set becomes completely empty
($\mathcal{M}(w) = \emptyset$),

the total loss drops to exactly 0.

The updates loop terminates, freezing the parameters at final, solid decision boundary".